

DURGA

SYSTEM AND DEVELOPMENT DOCUMENTATION

Frode Randers

2026-06-12

Contents

1	Introduction	3
2	System Overview	4
2.1	Repository-level components	4
2.2	Generated runtime shape	4
2.3	Canonical event model	5
3	Architecture Decisions	6
3.1	Canonical lifecycle stream	6
3.2	Visible generated runtime	6
3.3	Generated helper scripts	6
3.4	Kafka Streams as the default read model	6
3.5	Scope-aware subprocess handling	6
4	Execution Model	7
4.1	General mapping strategy	7
4.2	Core control-flow support	7
4.3	Tasks and routing	7
4.4	Boundary behavior	8
4.5	Subprocesses and event subprocesses	8
4.6	Current execution boundary	9
5	BPMN Support And Limits	10
5.1	Well-supported areas	10
5.2	Current semantic boundaries	10
5.3	Still outside the current focus	10
6	Monitoring and Observability	11
6.1	Canonical process-events topic	11
6.2	Kafka Streams topology	11
6.3	Query surface	11

7	Monitoring Topology	12
7.1	Purpose	12
7.2	Topology structure	12
7.3	State stores	12
7.4	Latency computation	13
7.5	Processing guarantees	13
7.6	Query access pattern	13
7.7	Relationship to demos	13
7.8	Current shape	14
8	Monitoring Interfaces	15
8.1	HTTP service	15
8.2	CLI client	15
8.3	Dashboard	16
8.4	How the interfaces fit together	16
8.5	What this enables in practice	16
9	Generated Projects	17
9.1	Generated files	17
9.2	Helper scripts	17
9.3	Development workflow	17
10	Deployment	18
10.1	Architecture	18
10.2	Local development & testing	18
10.3	Connection to Kafka	19
10.4	Topic provisioning	19
10.5	Test environment deployment	20
10.6	Production deployment	21
10.7	Monitoring in production	21
10.8	Configuration reference	22

11	Generated Code Extension Strategy	23
11.1	What is edited	23
11.2	What should remain generated	23
11.3	Regeneration strategy	23
12	Plugin Architecture.....	24
12.1	Plugin registry	24
12.2	Governed categories	24
12.3	Plugin interface	24
12.4	Reference implementations.....	24
12.5	BPMN binding	25
12.6	Generated executor.....	25
12.7	Current limitations	25
13	Generated Runtime Classes	26
13.1	Class families	26
13.2	Task services	26
13.3	Routing and event classes	26
13.4	Scope classes.....	26
14	Naming Conventions	27
14.1	Common naming patterns	27
14.2	Why this matters	27
14.3	Practical guidance.....	27
15	Generated Project Lifecycle	28
15.1	Two modes of use	28
16	Demo Flows And Helper Scripts.....	29
16.1	Purpose.....	29
16.2	Common generated scripts	29
16.3	Recommended learning loop.....	29

17	Configuration And Operations	30
17.1	Local runtime baseline	30
17.2	Important configuration surfaces	30
17.3	Operational expectations	30
17.4	Where state lives	30
17.5	Recommended local operator loop	30
18	Testing And Verification	32
18.1	Generator verification	32
18.2	Representative sample coverage	32
18.3	Core runtime contract verification	32
18.4	Monitoring verification	32
18.5	CI pipeline	33
18.6	What is not fully covered yet	33
19	Troubleshooting	34
19.1	Kafka UI logs connection failures at startup	34
19.2	Monitoring app says the state store is not queryable yet	34
19.3	A generated process compiles but does not advance	34
19.4	Boundary behavior does not appear to trigger	34
19.5	An external completion arrives after cancellation	34
19.6	Monitoring counts do not match raw topic intuition	35
19.7	The generated output looks harder to understand than expected	35
20	Known Design Tensions	36
20.1	Explicitness versus compactness	36
20.2	BPMN fidelity versus pragmatic Kafka mapping	36
20.3	Regeneration versus local customization	36
20.4	Exploration versus production hardening	36
21	BPMN Sample Catalog	37

22	Example Walkthrough	39
22.1	Chosen example	39
22.2	Generation step	39
22.3	What appears in the generated project.....	39
22.4	Runtime shape.....	39
22.5	How to explore the generated process	40
23	Current Status	41
23.1	Maturity	41
23.2	What is solid today	41
23.3	Important limits.....	41
23.4	Likely next decisions	41
23.5	Practical summary	42
24	Glossary.....	43

1 Introduction

Durga is a BPMN-driven Kafka scaffolding tool. It reads BPMN models, extracts an executable subset of process semantics, and generates Java and Kafka-oriented project skeletons that can be used to explore, test and refine workflow designs.

The project is not a finished workflow engine. It is best understood as a generator plus a local experimentation surface:

- BPMN models are parsed with Camunda,
- Java runtime skeletons are rendered with StringTemplate,
- Kafka topics and reactive-messaging bindings are generated from the process graph,
- supporting probes and demos are generated so the resulting process can be exercised locally,
- a separate Kafka Streams monitoring path materializes process state and operational views.

This manual describes the current system shape rather than an ideal future architecture. It focuses on what Durga generates today, what runtime model those generated artifacts implement, and where the present support boundary still ends.

The document is organized in that order. Section 2 describes the major subsystems. Section 4 explains the BPMN-to-Kafka execution mapping. Section 6 describes the canonical process-event and monitoring model. Section 9 describes the generated project surface, and Section 23 summarizes maturity, constraints and current gaps.

2 System Overview

Durga has three centers:

- a BPMN parsing and generation pipeline,
- a generated Kafka-oriented process runtime,
- a monitoring and reporting path built around canonical lifecycle events.

2.1 Repository-level components

Scaffolder Parses a BPMN model and renders a generated project (entry point: `org.gautelis.durga.tools.Bpm`

Generation support Model extraction and template coordination: `BpmnModelSupport`, `TaskRoutingGenerator`, `EventTemplateGenerator`, `SubProcessTemplateGenerator`, `GeneratedProjectSupport`.

Templates StringTemplate groups under `src/main/resources/templates/` define generated Java classes, scripts, YAML, Maven build files and shared runtime helpers.

Monitoring runtime A Kafka Streams topology consumes canonical lifecycle events and materializes state, counts, latency summaries and stuck-instance views.

2.2 Generated runtime shape

Each generated project contains:

- worker and routing services,
- a starter,
- a process orchestrator,
- helpers for user/manual task completion and failure testing,
- generated Kafka topic configuration,
- sample payloads and runnable scenario scripts.

Topics and handlers are deliberately visible so that BPMN concepts are projected concretely onto Kafka channels and lifecycle events.

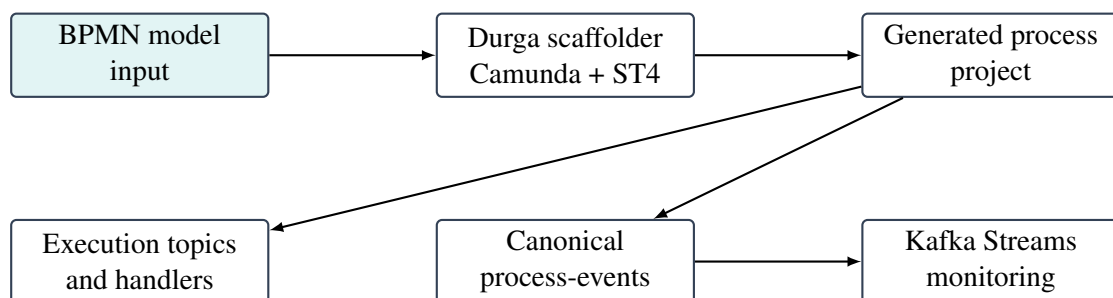


Figure 1: High-level flow from BPMN model to generated runtime and monitoring projections

2.3 Canonical event model

The shared canonical stream on `process-events` is the observability backbone for latest state per instance, counts by current state, latency summaries, stuck-instance detection, dashboards and query endpoints. Execution topics may be useful for routing visibility, but the monitoring model depends on explicit projections rather than raw topic inspection.

3 Architecture Decisions

3.1 Canonical lifecycle stream

The shared `process-events` topic carries normalized lifecycle events. Execution topics carry work between generated handlers, but they are not the authoritative query surface. State projections are built from lifecycle events because:

- execution topics represent transport history, not current truth,
- process instance state must be queryable independently of routing topology,
- counts, latency summaries and stuck-instance views are easier to derive from explicit lifecycle events than from task-topic inspection.

3.2 Visible generated runtime

Durga generates many explicit classes instead of a single hidden execution engine. The output is more verbose, but far easier to inspect — making BPMN-to-Kafka mapping concrete.

3.3 Generated helper scripts

Generated probes, scenario runners and manual interaction scripts let users exercise the runtime without writing custom producers and consumers.

3.4 Kafka Streams as the default read model

The monitoring side consumes one canonical stream and materializes queryable state. It solves the local architectural read-model problem directly rather than every production reporting problem.

3.5 Scope-aware subprocess handling

Subprocess support uses explicit entry and completion services because scope boundaries matter when cancellation, failure, escalation and event subprocesses are involved.

4 Execution Model

4.1 General mapping strategy

Durga maps a BPMN graph into a bounded Kafka runtime surface. In the current implementation, each supported BPMN element is converted into either:

- a generated service consuming from one topic and producing to one or more topics,
- a scope handler that turns subprocess semantics into lifecycle and routing events,
- or an external helper path that lets a human or another tool supply the missing runtime action.

The model is intentionally explicit. Instead of hiding process behavior behind one opaque runtime, the generated project exposes the process topology as concrete Kafka topics, generated services and helper scripts.

4.2 Core control-flow support

The executable subset currently includes:

- start events,
- service, user and manual tasks,
- exclusive and parallel gateways,
- call activities,
- explicit end-event completion,
- embedded subprocesses,
- several categories of boundary events,
- event subprocesses for selected start types.

The important distinction is that not all BPMN elements are treated the same way. A service task auto-completes in the generated worker model, while a user or manual task enters a waiting state and must be advanced through a generated completion helper.

4.3 Tasks and routing

Service tasks are generated as workers consuming `%processId%_%taskId%_input` and producing `%processId%_%taskId%_output`.

User and manual tasks consume their input topic, emit an `ACTIVITY_ENTERED` lifecycle event, and then wait for an external completion event generated via helper scripts.

Exclusive gateways are generated as routing services that evaluate simple conditions against the event payload and emit the selected next activity. Parallel splits fan out to several downstream inputs. Parallel joins use the generated process-state store to wait for the required set of incoming completions before forwarding the flow.

4.4 Boundary behavior

Boundary timer, error and escalation handlers are generated from BPMN boundary events and consume the canonical lifecycle stream on `process-events-monitor`. The current boundary implementation supports:

- interrupting and non-interrupting timer boundaries,
- interrupting and non-interrupting error boundaries,
- interrupting and non-interrupting escalation boundaries,
- task-attached and subprocess-attached variants.

Interrupting boundaries now have behavioral effect, not just monitoring effect. Generated handlers consult the shared cancellation registry so that canceled work is suppressed rather than continuing to emit normal downstream progress.

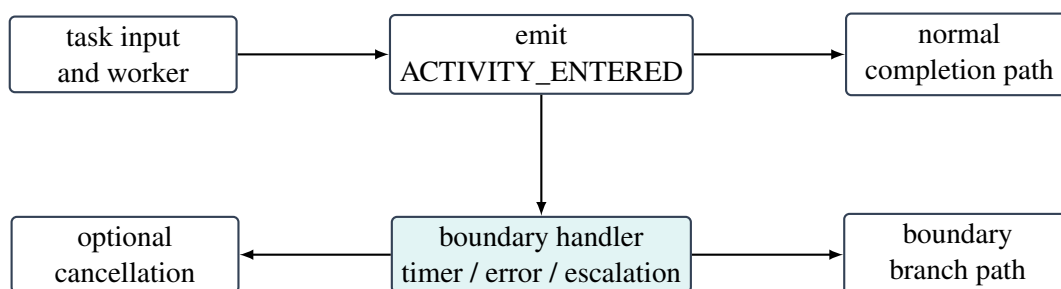


Figure 2: Generated boundary handling separates normal flow from boundary-triggered flow

4.5 Subprocesses and event subprocesses

Embedded subprocesses are generated with explicit entry and completion services. These services emit subprocess-level lifecycle events and route into and out of the internal subprocess graph. Nested subprocesses are supported with the same pattern.

Event subprocesses are currently implemented as scoped handlers rather than as a full general-purpose BPMN scope engine. The supported start types are:

- message,
- signal,
- timer inside an embedded subprocess scope,
- error inside an embedded subprocess scope,
- escalation inside an embedded subprocess scope.

Message and signal event subprocesses consume from dedicated external topics. Timer, error and escalation event subprocesses instead listen to scoped lifecycle signals on `process-events-monitor`. Interrupting event subprocesses emit cancellation for the enclosing scope before starting the event subprocess branch. Non-interrupting event subprocesses branch without canceling the parent flow.

4.6 Current execution boundary

The current execution boundary is broad enough for local process experimentation, but it is not full BPMN coverage. Inclusive gateways, complex gateways, BPMN-native data objects, and more advanced scope semantics are still out of scope. That boundary should be treated as a deliberate implementation limit, not as an accidental omission.

5 BPMN Support And Limits

Durga supports a pragmatic BPMN subset that is broad enough for exploration and code generation, but not yet a complete BPMN execution engine.

5.1 Well-supported areas

The strongest current coverage includes:

- start events and explicit end completion handling,
- service, user, manual, send, receive, script and business rule tasks,
- XOR, inclusive (OR) and parallel gateway routing,
- intermediate timer, message and signal events,
- task and subprocess boundary timer, error and escalation handling,
- call activities,
- embedded subprocesses including nested subprocesses,
- event subprocesses for message, signal, timer, error and escalation starts.

5.2 Current semantic boundaries

Some behaviors are supported only to the level needed for explicit generated runtimes, not to full BPMN-engine fidelity:

- cancellation propagation is pragmatic rather than globally token-precise,
- subprocess scope handling is explicit but still lighter than a full BPMN execution scope manager,
- monitoring now includes dedicated current-state indexes and coarse trend buckets, but it is still lighter on long-term analytical history than on current-state inspection,
- generated runtimes are optimized for understanding and experimentation rather than minimalism.

5.3 Still outside the current focus

The main areas still outside the project's current center of gravity are:

- full production-grade workflow-engine semantics,
- richer compensation and advanced BPMN exception orchestration,
- large-scale operational hardening beyond local and exploratory usage,
- a generalized multi-node monitoring and query deployment model.

The repository coverage matrix remains the most detailed element-by-element reference, but the overall design intent is simple: support enough BPMN to make the Kafka mapping useful, while keeping the generated runtime explicit and teachable.

6 Monitoring and Observability

6.1 Canonical process-events topic

Durga emits normalized lifecycle events on the shared topic `process-events`. Generated workers, gateways, subprocess handlers, boundary handlers and helper publishers all emit events describing process progress, waiting states, completion, failure, escalation and cancellation.

This separates execution transport from queryable state. Execution topics remain useful for routing, but the monitoring model depends on projections built from explicit lifecycle events.

6.2 Kafka Streams topology

A Kafka Streams monitoring topology consumes `process-events` and materializes:

- latest process state by instance,
- counts by current state,
- latency summaries,
- stuck-instance views.

The main entry point is `org.gautelis.durga.monitoring.ProcessMonitoringApp`. The projected instance model is `ProcessStateView`. The query side is exposed through an HTTP service and a CLI client.

6.3 Query surface

The monitoring service exposes endpoints for health, instance lookup, counts, latency, stuck instances and a lightweight dashboard. The local operator loop is:

1. generate a process,
2. publish scenarios or task inputs,
3. observe lifecycle events,
4. query current state and latency,
5. iterate on the BPMN model or generated runtime.

7 Monitoring Topology

7.1 Purpose

The monitoring subsystem is a separate runtime concern from the generated process execution graph. Generated handlers emit canonical lifecycle events; the monitoring topology consumes those events and turns them into queryable operational views.

Execution topics show how work is routed. The monitoring topology answers:

- where is a specific process instance now,
- how many instances are in a given state,
- which activities are taking the longest,
- which instances appear stuck,
- what are the coarse trend counts for starts, completions, and failures.

7.2 Topology structure

The entry point is `ProcessMonitoringTopology.buildTopology()`. It consumes `process-events`, filters null keys and values, and forks the stream into five parallel sub-topologies, each with a dedicated local state store:

Instance state Aggregates events into `ProcessStateView` per instance. Materialized in `process-state-store`, output to `process-state`. A filtered subset (active instances) is output to `process-active-state`.

State counts Regroups state views by `processId:state` key and counts instances. Materialized in `process-state-counts-store`, output to `process-state-counts`.

Activity latency Feeds events through a custom `ActivityLatencyProcessor` that records `ACTIVITY_ENTERED` timestamps in a persistent key-value store (`process-activity-entry-store`) and, on later completion or termination events, emits duration deltas. These deltas are aggregated into rolling `ActivityLatencyStats` per activity, then mapped to `ActivityLatencySummary` and output to `process-latency`.

Trends Filters to `PROCESS_STARTED`, `PROCESS_COMPLETED`, and `PROCESS_FAILED` events, maps them to hourly trend bucket keys, counts per bucket, and outputs `ProcessTrendPoint` records to `process-trends`. Materialized in `process-trends-store`.

Queryable replicas Five `builder.globalTable()` calls consume the output topics above, each materialized as a global table with a `-global-store` suffix. These are full replicas on every monitoring instance and are what the query service reads.

7.3 State stores

The topology maintains ten state stores:

Store	Type	Purpose
<code>process-state-store</code>	Local KTable aggregate	Per-instance latest <code>ProcessStateView</code>
<code>process-state-counts-store</code>	Local KTable count	Counts per <code>processId:state</code> key

Store	Type	Purpose
process-latency-store	Local KTable aggregate	Rolling ActivityLatencyStats per activity
process-activity-entry-store	Persistent KV	Timestamps for the latency temporal join
process-trends-store	Local KTable count	Trend bucket counts
*-global-store (5×)	Global table	Full replicas for HTTP/CLI query access

The five local stores drive computation — aggregation, counting, and temporal joins. The five global stores serve queries. The output topics (`process-state`, `process-state-counts`, `process-active-state`, `process-latency`, `process-trends`) decouple computation from query access: any monitoring instance can spin up and receive a full read replica without recomputing.

7.4 Latency computation

Activity latency uses a temporal join pattern without windowed stream-stream joins. The `ActivityLatencyProcessor` stores each `ACTIVITY_ENTERED` event's timestamp under the key `instanceId:activityId` in the persistent `process-activity-entry-store`. When a later `ACTIVITY_COMPLETED`, `ACTIVITY_ESCALATED`, `ACTIVITY_CANCELLED`, `GATEWAY_TAKEN`, `PROCESS_COMPLETED`, or `PROCESS_FAILED` event arrives for the same key, the processor computes the duration and emits an `ActivityLatencyDelta`.

These deltas are grouped by a canonical activity key (`processId`, `version`, `activityId`, event type) and aggregated into `ActivityLatencyStats` — a running count, sum, min, and max — then mapped to `ActivityLatencySummary` for output.

7.5 Processing guarantees

The topology is configured with `AT_LEAST_ONCE` semantics. This trades exactly-once guarantees for higher throughput and lower latency. State views may transiently reflect duplicate or stale updates, which is acceptable for a monitoring read model. The state directory is configured to `target/kafka-streams-state` — suitable for local development, but should be made configurable for production use.

7.6 Query access pattern

The query service (`ProcessMonitoringQueryService`) accesses only the global table stores via `KafkaStreams.store()`. It uses point lookups for instance state and full-store scans for counts, latency, trends, and stuck detection — filtering in memory by process ID where needed. No interactive queries or cross-instance RPC is required, since global tables replicate the entire content to every monitoring instance.

7.7 Relationship to demos

Repository-level demo publishers and scenario runners emit synthetic lifecycle events directly to `process-events`, making the monitoring subsystem independently testable without a full generated process runtime.

7.8 Current shape

The monitoring path is now stronger than the original local-only read model:

- queryable state is replicated from monitoring topics into global read-side stores, so each instance carries a full local copy of the monitoring model,
- latency summaries are maintained incrementally via a custom processor rather than derived from scanning latest-instance state,
- active-instance queries use a dedicated active-state index instead of the full state projection,
- coarse history is maintained as bucketed trend counts for process starts, completions, and failures.

History is intentionally coarse — Durga supports basic trend reporting but is optimized more for current-state inspection than long-retention analytical reporting.

8 Monitoring Interfaces

The monitoring subsystem is exposed through three interfaces that sit on top of the same Kafka Streams state:

- an HTTP query service,
- a small command-line client,
- a lightweight browser dashboard.

This split is intentional. The HTTP service is the primary integration surface, the CLI is the fastest way to inspect state from a terminal, and the dashboard is the fastest way to watch the system move.

8.1 HTTP service

The monitoring HTTP service is started by the monitoring application. Its main entry point is:

```
org.gautelis.durga.monitoring.ProcessMonitoringApp
```

The query layer is implemented by the repository classes `ProcessMonitoringQueryService` and `ProcessMonitoringHttpServer`.

The main endpoints are:

- `/health` for Kafka Streams readiness,
- `/instances/{processInstanceId}` for the latest instance view,
- `/processes/{processId}/counts` for process-specific current-state counts,
- `/processes/{processId}/latency` for activity latency summaries,
- `/processes/{processId}/trends` for bucketed history of starts, completions and failures,
- `/counts` for the global state-count view,
- `/stuck` for stuck-instance detection,
- `/dashboard` serving a lightweight browser dashboard that polls the same query surface.

The service is intentionally narrow. It is not trying to be a full workflow management API. Its role is to expose the read model produced by the monitoring topology.

8.2 CLI client

The command-line companion is `org.gautelis.durga.monitoring.ProcessMonitoringClient`. It exists because interactive inspection is common during development and BPMN exploration.

The main client operations are:

- `health`,
- `instance <processInstanceId>`,
- `counts [processId]`,
- `latency <processId>`,

- `stuck [processId] [olderThanSeconds]`.

The CLI is useful when the user is already working with the generated helper scripts or with local Kafka infrastructure from a terminal. It keeps the entire observation loop close to the generated runtime.

8.3 Dashboard

The browser dashboard is served from `/dashboard`. It polls the same HTTP endpoints that the CLI uses and presents:

- Kafka Streams health,
- counts by current state,
- latency summaries,
- stuck instances,
- selected instance details.

The dashboard is intentionally simple. It avoids a separate frontend build and keeps the monitoring surface close to the backend implementation. That makes it suitable for local architectural exploration, generated-process demos, and small-scale testing.

8.4 How the interfaces fit together

The three interfaces are not separate products. They are different views over the same materialized state:

1. generated or demo events are published to `process-events`,
2. Kafka Streams materializes state and counts,
3. the HTTP layer exposes that materialized state,
4. the CLI and dashboard consume the same HTTP surface.

This layering is important because it keeps the monitoring model coherent. The dashboard does not contain business logic that differs from the CLI, and the CLI does not bypass the query model. The read model remains single-sourced.

8.5 What this enables in practice

Taken together, the interfaces support a practical operator and learner workflow:

- publish a scenario or task outcome,
- watch `process-events`,
- query one instance in detail,
- inspect counts and latency summaries,
- switch to the dashboard when a broader view is helpful.

That workflow is one of the reasons Durga is usable as an exploration environment. Without these interfaces, the generated BPMN runtime would be much harder to evaluate as a concrete Kafka-based process system.

9 Generated Projects

9.1 Generated files

The generated output contains:

- Java runtime classes under `src/main/java/org/gautelis/durga/generated/`,
- a generated `pom.xml`,
- merged Kafka channel configuration in `application.yml`,
- a topic-creation script,
- sample payload data in `task-payloads.json`,
- a generated README,
- helper scripts for demos, inputs, completions, failures, escalations and observers.

9.2 Helper scripts

Generated scripts include:

- `demo-scenario.sh`,
- `send-task-input.sh`,
- `complete-task.sh`,
- `fail-task.sh`,
- `escalate-task.sh`,
- `complete-call-activity.sh`,
- `send-message-event.sh`,
- `send-signal-event.sh`,
- `watch-process-events.sh`,
- `watch-task-output.sh`.

9.3 Development workflow

1. generate the project from a BPMN model,
2. inspect `summary.json`, topic config and helper scripts,
3. build the generated project independently,
4. run a starter or a scenario,
5. observe progress through `process-events` and the monitoring views,
6. adjust the BPMN model and regenerate.

10 Deployment

Durga-generated projects are plain Java applications that connect to Kafka. They can be deployed as JARs, Docker containers, or Kubernetes workloads with no Durga-specific runtime dependencies beyond a Kafka cluster.

10.1 Architecture

Each generated process instance is a set of Quarkus-based workers (one per BPMN task) communicating through Kafka topics. There is no central orchestrator at runtime — workers consume from input topics and produce to output topics independently. This means:

- every worker is horizontally scalable via Kafka consumer groups,
- deployment is stateless (all state lives in Kafka topics and Streams stores),
- the generated shaded JAR is self-contained (all dependencies bundled).

10.2 Local development & testing

The recommended local loop uses Docker Compose for Kafka and direct JVM execution for the pipeline:

```
# Start Kafka
cd setup && docker compose up -d

# Build and run the pipeline
cd generated/<project>
START_KAFKA=false BOOTSTRAP=localhost:9094 ./run-local.sh
```

For iterative development, keep Kafka running and rebuild the pipeline:

```
mvn -q clean package -DskipTests
java -cp target/<project>-all.jar \
  org.gautelis.durga.generated.<Process>Starter localhost:9094
```

To test with the monitoring dashboard:

1. Start Kafka as above.
2. Start the monitoring app:

```
cd setup
START_KAFKA=false ./demo-monitoring.sh
```

3. Run pipeline scenarios via the generated helper scripts.
4. Open <http://localhost:18081/dashboard> for the Svelte dashboard or <http://localhost:8081> for the Quarkus REST API.

10.3 Connection to Kafka

Kafka bootstrap servers are configured through a chain of fallbacks:

1. JVM system property `kafka.bootstrap.servers`.
2. Environment variable `KAFKA_BOOTSTRAP_SERVERS`.
3. Hardcoded default `localhost:9094`.

In Docker and Kubernetes, set `KAFKA_BOOTSTRAP_SERVERS` as a container environment variable. The generated `Dockerfile` passes it to the JVM:

```
ENTRYPOINT ["java", "-Xms64m", "-Xmx256m", \
  "-Dkafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:-localhost:9094}", \
  "-jar", "/app/pipeline.jar"]
```

10.4 Topic provisioning

Kafka topics must exist before the pipeline starts. There are three approaches, each supported by the generated artifacts.

10.4.1 Pre-creation via shell script

The generated `topics.sh` creates all required topics using the Kafka CLI tools bundled with the Kafka distribution:

```
KAFKA_BOOTSTRAP_SERVERS=kafka-broker:9092 ./topics.sh
```

This is suitable for development, staging, and manual production setups. For automated environments, run `topics.sh` as an init container or a CI/CD pipeline step before deploying the workers.

10.4.2 Init container

For Kubernetes deployments where the Kafka broker shares the cluster, add an init container to the generated `k8s.yml`:

```
initContainers:
- name: topic-init
  image: bitnami/kafka:latest
  command: ['/bin/sh', '-c']
  args:
  - |
    /opt/bitnami/kafka/bin/kafka-topics.sh --create \
      --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
      --topic <name> --partitions 3 --replication-factor 1
```

10.4.3 Kafka operator (Strimzi)

In Kubernetes environments running the Strimzi operator, topics can be declared as `KafkaTopic` custom resources. A generated `kafka-topics.yml` CRD manifest can be applied alongside the Deployment:

```
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: <process>_<task>_input
  labels:
    strimzi.io/cluster: my-kafka
spec:
  partitions: 3
  replicas: 1
---
```

The scaffolder currently generates `topics.sh`; Strimzi CRD generation can be added as an optional output format.

10.5 Test environment deployment

A Docker Compose-based test environment is the fastest way to validate a pipeline end-to-end. The repository's `setup/docker-compose.yml` provides a single-node Kafka broker and Kafka UI.

Add a `docker-compose.test.yml` to the generated project:

```
version: "3"
services:
  pipeline:
    build: .
    environment:
      KAFKA_BOOTSTRAP_SERVERS: kafka:9092
    depends_on:
      - kafka
  kafka:
    image: bitnami/kafka:latest
    environment:
      KAFKA_CFG_NODE_ID: 1
      KAFKA_CFG_PROCESS_ROLES: broker,controller
      KAFKA_CFG_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_CFG_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_CFG_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CFG_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
```

Run with:

```
docker compose -f docker-compose.test.yml up --build
```


10.6 Production deployment

The generated `deploy.sh` supports three targets:

JAR `./deploy.sh` — builds the shaded JAR and runs locally. Suitable for VMs and bare-metal deployments.

Docker `DOCKER=true ./deploy.sh` — builds a Docker image tagged `<processId>:latest`.

Kubernetes `DOCKER=true K8S=true ./deploy.sh` — builds the image and applies the generated `k8s.yml`.

10.6.1 Docker image

The generated Dockerfile uses `eclipse-temurin:21-jre-alpine`, runs as a non-root user, and includes a process-based health check:

```
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S durga && adduser -S durga -G durga
USER durga
COPY target/*-all.jar /app/pipeline.jar
HEALTHCHECK --interval=30s ...
ENTRYPOINT ["java", "-Xms64m", "-Xmx256m", \
    "-Dkafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:-...}", \
    "-jar", "/app/pipeline.jar"]
```

10.6.2 Kubernetes

The generated `k8s.yml` provides:

- A Deployment with 2 replicas (configurable), resource requests and limits (128Mi/512Mi memory, 100m/500m CPU),
- A Service exposing port 8080,
- Exec-based liveness and readiness probes.

Scaling is done via `kubectl scale` or the `replicas` field:

```
kubectl scale deployment/<processId> --replicas=4
```

Maximum parallelism per consumer group is bounded by the number of Kafka partitions for each topic. Configure partitions in `topics.sh` before provisioning, then scale workers up to the partition count.

10.7 Monitoring in production

The monitoring topology (`ProcessMonitoringApp`) is a separate deployment that consumes the canonical `process-events` topic and materializes queryable state. In production:

- Deploy the monitoring app as a companion service alongside the pipeline (same Kafka cluster, different consumer group).
- The Quarkus REST API and Svelte dashboard provide operational visibility — instance counts, latency summaries, stuck-instance detection, trend reporting.
- The monitoring topology uses global tables: every instance has a full local replica of the monitoring model. Any instance can answer all queries without cross-instance RPC.

10.8 Configuration reference

Variable	Default	Description
KAFKA_BOOTSTRAP_SERVERS	localhost:9094	Kafka broker list
HTTP_PORT	8080	Quarkus HTTP port
PROFILE	dev	Quarkus configuration profile
START_KAFKA	false	Start Docker Compose Kafka before running
DOCKER	false	Build Docker image in deploy script
K8S	false	Apply Kubernetes manifests in deploy script

11 Generated Code Extension Strategy

Generated code is a starting point. The BPMN model is the primary design source; generated code is scaffolding expected to be completed and adapted.

11.1 What is edited

Common edits in generated projects:

- task business logic,
- XOR decision conditions,
- payload handling and validation,
- integration calls inside service-style workers,
- operational configuration values.

11.2 What should remain generated

Keep these close to the generated structure:

- topic and channel topology,
- lifecycle event emission shape,
- probe script surface,
- subprocess and boundary wiring,
- monitoring-facing event semantics.

11.3 Regeneration strategy

Preserve the overall generated runtime structure while filling in task-specific behavior. When the BPMN model changes substantially, regenerate and re-apply focused logic changes rather than manually rewiring the generated topology.

12 Plugin Architecture

Durga supports a plugin-driven data pipeline model where BPMN `ServiceTask` elements can be annotated with `camunda:plugin` attributes that resolve to registered, governed data processing components.

12.1 Plugin registry

Plugins are defined by YAML descriptors under `plugins/` and indexed by `plugins/catalog.yml`. Each descriptor declares:

- `id` — unique plugin identifier (used in `camunda:plugin`),
- `name / version` — human-readable identifier,
- `category` — governed taxonomy (transform, validate, enrich, route, aggregate, sink),
- `status` — lifecycle state (experimental, stable, deprecated, retired),
- `implementation` — Java class implementing `org.gautelis.durga.plugins.Plugin`,
- `config` — typed configuration schema (field types, defaults, allowed values).

12.2 Governed categories

Each plugin category enforces a contract: topic naming, lifecycle events, and error channel conventions. The scaffolder validates that generated executors follow the category contract.

Category	Contract
transform	1:1 input→output, <code>ACTIVITY_COMPLETED</code> , dlq on failure
validate	Output + invalid branch, <code>ACTIVITY_ESCALATED</code> on invalid
enrich	Input→enriched output, may block on external lookup
route	Input→N downstream channels, <code>GATEWAY_TAKEN</code> lifecycle
aggregate	Windowed counting, <code>ACTIVITY_ENTERED</code> on window start
sink	Terminal output, no downstream routing

12.3 Plugin interface

All plugins implement `org.gautelis.durga.plugins.Plugin`:

```
Stringexecute(Stringpayload,Stringconfig)throwsException
```

Generated executor classes instantiate the plugin at field level and call `plugin.execute(payload, config)` for each message. The config string comes from the BPMN `camunda:pluginConfig` property.

12.4 Reference implementations

The repository ships with five reference plugins that form the initial standard library:

JSON Transform Dot-notation field remapping.

Field Filter Whitelist/blacklist with flatten.

KV Enricher Inline key-value lookup and merge.

Field Router Value-based routing to named channels.

Window Counter Time-window counting with grouping.

12.5 BPMN binding

Plugin-annotated tasks use Camunda extension properties on `ServiceTask` elements:

```
<bpmn:serviceTask id="transform" name="Transform Data">
  <bpmn:extensionElements>
    <camunda:properties>
      <camunda:property name="plugin" value="json-transform" />
      <camunda:property name="pluginConfig" value="name, email" />
    </camunda:properties>
  </bpmn:extensionElements>
</bpmn:serviceTask>
```

Without a `camunda:plugin` attribute, `ServiceTask` elements fall back to the existing generic worker generation.

12.6 Generated executor

The scaffolder resolves the plugin descriptor, validates it against the registry, and generates a concrete executor class using the `pluginExecutorClass` template. The executor:

- imports the plugin implementation class and the `Plugin` interface,
- instantiates the plugin as a private field,
- delegates each incoming message to `plugin.execute()`,
- emits `ACTIVITY_COMPLETED` on success,
- routes failures to a dead-letter topic (`%task%_dlq`) with structured error records,
- respects `ScopeCancellationRegistry` for interrupting boundary support.

12.7 Current limitations

- Plugin descriptors are read from the filesystem or classpath at scaffold time — there is no runtime plugin discovery or hot-reload.
- Config validation at scaffold time is not yet implemented — invalid configs are caught at runtime by the generated executor.
- The plugin catalog must be maintained manually; there is no registry publishing or version-resolution tooling.

13 Generated Runtime Classes

Each generated project contains Java classes under `src/main/java/org/gautelis/durga/generated/`. These classes map BPMN constructs onto Kafka consumers, producers and lifecycle events rather than hiding behind an opaque engine.

13.1 Class families

- **Starter classes** publish the initial lifecycle event and first task input.
- **Task services** handle BPMN tasks. Service-task handlers auto-complete after their work step; user and manual task handlers enter a waiting state and await explicit completion, failure or escalation input.
- **Gateway services** handle XOR and parallel routing decisions, including fan-out and join behavior.
- **Event handlers** implement timer, message and signal behavior for intermediate events and boundary-event reactions.
- **Call-activity helpers** model request and reply behavior for call activities.
- **Subprocess scope services** emit subprocess entry and completion lifecycle events and coordinate subprocess-local routing.
- **Process orchestrator** observes terminal flows and emits process completion.
- **Probe helpers** provide generated publishers and watchers for command-line exploration.

13.2 Task services

Generated task services consume a task input topic, emit `ACTIVITY_ENTERED`, perform the generated action or enter a wait state, then emit `ACTIVITY_COMPLETED`, `FAILED`, `ESCALATED` or `CANCELLED` before forwarding to the next topic or boundary path.

13.3 Routing and event classes

Gateways turn BPMN branching into explicit topic routing. Timer, message and signal handlers make delays and external triggers concrete. Boundary handlers observe task or scope state and branch into exceptional or reminder flows when trigger conditions are met.

13.4 Scope classes

Subprocess scope services emit subprocess-level entered and completed events, mediate flow into and out of the subprocess graph, propagate failure, escalation and cancellation at scope level, and support nested subprocess structure. Event subprocess handlers model subprocess-internal event-driven branches separately from the main task path.

14 Naming Conventions

Durga relies heavily on predictable names because the generated topology is intentionally visible. Naming is therefore part of the system design, not only a cosmetic detail.

14.1 Common naming patterns

The generator generally follows these patterns:

- process identifiers come from BPMN process ids,
- task topics use process and task identifiers,
- generated classes use process and BPMN element names converted to Java class names,
- helper scripts use short imperative file names such as `complete-task.sh`,
- monitoring topics use shared names such as:

```
process-events  
process-state  
process-state-counts
```

14.2 Why this matters

Consistent names make it easier to:

- inspect topic traffic in Kafka tools,
- map BPMN elements to generated Java handlers,
- reason about subprocess and boundary scope behavior,
- debug routing and completion behavior quickly.

14.3 Practical guidance

For the cleanest generated output, BPMN ids should be stable, readable and explicit. If BPMN models use unclear or heavily tool-generated ids, the generated Kafka and Java surface becomes harder to read as well.

15 Generated Project Lifecycle

Generated projects are used iteratively, not as one-off code dumps:

1. model a BPMN process,
2. scaffold a project from that model,
3. inspect generated topics, classes and helper scripts,
4. run demo flows or manual probes,
5. observe the lifecycle stream and monitoring views,
6. revise the BPMN model and regenerate.

15.1 Two modes of use

- as a learning scaffold, where the value is seeing how BPMN maps onto Kafka,
- as a starting point for a real implementation, where the generated code becomes a base that is extended, completed and hardened.

The BPMN model remains the primary design artifact. Generated output is readable, runnable and observable by design.

16 Demo Flows And Helper Scripts

16.1 Purpose

Generated scripts provide runnable examples of the Kafka topology, reduce the amount of manual producer and consumer work needed during exploration, and make BPMN semantics visible through actions such as completion, failure, escalation and observation.

16.2 Common generated scripts

The exact set depends on the BPMN model. Common script categories include:

- `demo-scenario.sh` for whole-process happy, stuck or failed runs,
- `send-task-input.sh` for manual injection into one task input topic,
- `complete-task.sh`, `fail-task.sh` and `escalate-task.sh` for human or exceptional task outcomes,
- `complete-call-activity.sh` for returning from a generated call activity,
- `send-message-event.sh` and `send-signal-event.sh` for external event stimulation,
- `watch-process-events.sh` and `watch-task-output.sh` for runtime observation.

16.3 Recommended learning loop

1. generate a project from one small BPMN sample,
2. create the Kafka topics and start the generated runtime,
3. run `watch-process-events.sh`,
4. execute `demo-scenario.sh happy`,
5. repeat with a failure, escalation or timeout-oriented script,
6. inspect the monitoring endpoints or dashboard after each run.

This loop exposes the difference between nominal and exceptional flow with very little setup. The execution topics carry work, but the canonical `process-events` stream carries the lifecycle truth.

17 Configuration And Operations

17.1 Local runtime baseline

The normal local environment consists of:

- the bundled Kafka setup under `setup/`,
- generated topic configuration in the scaffolded project,
- the optional monitoring app and dashboard.

The repository is intentionally optimized for a local single-broker loop so the generated processes and monitoring projections can be explored quickly.

17.2 Important configuration surfaces

The main configuration locations are:

- repository-level runtime defaults in `src/main/resources/application.yml`,
- generated-project channel configuration in the scaffolded `application.yml`,
- local Kafka infrastructure in `setup/docker-compose.yml`,
- generated topic setup scripts in scaffolded output directories.

17.3 Operational expectations

In normal local startup:

- Kafka UI may log temporary connection failures before the broker is ready,
- generated runtimes rely on explicit topics and channels rather than implicit discovery,
- the monitoring app may report a transitional Streams state before its stores become queryable.

17.4 Where state lives

Durga uses several layers of state:

- transport history on execution topics,
- canonical lifecycle history on `process-events`,
- materialized latest state and counts in Kafka Streams stores and corresponding topics,
- local waiting and cancellation bookkeeping inside some generated handlers.

17.5 Recommended local operator loop

For practical local work:

1. start Kafka,

2. build the repository or generated project,
3. create topics,
4. run the generated process or a demo publisher,
5. start the monitoring app if query visibility is needed,
6. inspect lifecycle events, counts and latency.

18 Testing And Verification

The repository now has a meaningful regression net around the scaffolder, the monitoring path, and the core runtime contracts, even though it is still not a full end-to-end production validation environment.

18.1 Generator verification

Generator verification currently has three layers:

- dry-run tests that treat scaffold summaries and previews as a contract,
- generation tests that write selected projects to temporary directories and check the resulting files and configuration,
- compile-level tests that run Maven on representative generated projects.

This matters because the generator is split across BPMN parsing logic, StringTemplate groups and multiple helper generators. Simple unit tests are not enough on their own.

18.2 Representative sample coverage

The verification set intentionally spans several BPMN areas, including:

- message and signal events,
- timers and boundaries,
- subprocesses and nested subprocesses,
- call activities,
- event subprocesses.

18.3 Core runtime contract verification

Runtime contracts are validated through additional unit test coverage:

- `ProcessEvent` serialization round-trips and compact-constructor inference,
- `ProcessState` serialization round-trips,
- `ScopeCancellationRegistry` cancellation tracking, merging, clearing and null-guard behavior,
- expanded `ProcessStateView` projection tests covering cancellation, escalation, process completion, gateway events, multiple concurrent activities, null-activity edges and state-key generation.

18.4 Monitoring verification

The monitoring side is validated through:

- unit tests around the state-projection model,
- local startup and packaging checks,

- demo publishers and scenario runners that can drive the monitoring topology without a fully generated process runtime.

18.5 CI pipeline

A GitHub Actions CI workflow runs `mvn verify` on push and pull requests to the main branch using JDK 21, ensuring the complete test suite passes on every change.

18.6 What is not fully covered yet

The remaining verification gaps are mostly at the integrated runtime level:

- full end-to-end process executions across all BPMN features,
- broader load or concurrency-oriented behavior,
- stronger operational testing of multi-instance monitoring scenarios.

For the current project phase, the test strategy is therefore strongest on generator correctness and local runtime exploration, and lighter on production-style operational proof.

19 Troubleshooting

Durga is now runnable enough that practical failure modes matter. The most common problems are not subtle generator bugs, but startup timing, missing topics, incomplete task logic, or the difference between execution state and monitoring state.

19.1 Kafka UI logs connection failures at startup

This is usually normal in the local setup. Kafka UI often starts before the broker is fully ready and logs transient connection failures. If the broker then comes up and the errors stop, this is not itself a Durga problem.

19.2 Monitoring app says the state store is not queryable yet

This normally means Kafka Streams has started but the state stores are not yet ready for queries. The correct response is usually to wait briefly and retry rather than to assume the topology is broken immediately.

19.3 A generated process compiles but does not advance

The most common causes are:

- topics were not created,
- the relevant generated application is not running,
- an XOR branch condition still contains placeholder logic,
- a user or manual task entered a waiting state and needs explicit completion,
- the process instance was cancelled by a boundary or interrupting event subprocess.

19.4 Boundary behavior does not appear to trigger

Check these points first:

- the triggering lifecycle event is actually emitted,
- the boundary is interrupting or non-interrupting as intended,
- the attached task or subprocess reached the expected state,
- failure or escalation was emitted through the generated helper path rather than by bypassing the lifecycle model.

19.5 An external completion arrives after cancellation

The generated runtime now suppresses much of this post-cancel progress, but late events can still be confusing during debugging. In such cases, the canonical `process-events` stream is the best source of truth: check whether cancellation happened before the late completion arrived.

19.6 Monitoring counts do not match raw topic intuition

This is usually a model misunderstanding rather than a bug. Execution topics contain history and routing traffic. The monitoring counts are derived from the latest projected instance state, which is the intended read model for “how many instances are in state X right now”.

19.7 The generated output looks harder to understand than expected

This often comes from BPMN naming. Unclear BPMN ids produce unclear Java classes, topics and helper artifacts. Cleaning up the BPMN model identifiers usually improves the generated runtime surface more than post-processing the generated files.

20 Known Design Tensions

20.1 Explicitness versus compactness

The generated runtime is larger than a hidden workflow runtime would be. This is accepted because the BPMN-to-Kafka mapping is visible, making the system easier to inspect and debug.

20.2 BPMN fidelity versus pragmatic Kafka mapping

BPMN semantics are projected through explicit Kafka handlers and topics rather than a full engine. Some semantics are intentionally pragmatic rather than exhaustive.

20.3 Regeneration versus local customization

Generated projects invite local edits, but the BPMN model remains the primary artifact. When the model changes substantially, regeneration with re-application of focused logic changes is preferred over manual rewiring of the generated topology.

20.4 Exploration versus production hardening

The system is strong enough for architectural exploration, but not every part is optimized for hardened production operation. The current focus remains clarity, runnable demos, and generator correctness.

21 BPMN Sample Catalog

The repository contains a growing set of BPMN models under `src/test/resources/bpmn`. They serve two purposes:

- they are regression inputs for the scaffolder and its tests,
- they are concrete learning models for understanding one BPMN feature at a time.

The samples are intentionally small. Most of them isolate one feature rather than trying to model a realistic end-to-end business process in full detail.

Model	Primary focus
<code>invoice_receipt.bpmn</code>	Baseline process with start, service work, review, approval or rejection, and terminal notification. This is the best first model to inspect.
<code>order_fulfillment.bpmn</code>	Legacy process sample kept as an additional reference model. It is useful for checking that generator changes are not too tightly coupled to the invoice-oriented examples.
<code>invoice_receipt_reminder.bpmn</code>	Intermediate timer catch event between normal process steps. Demonstrates delay-driven routing without a boundary event.
<code>invoice_message_exchange.bpmn</code>	Intermediate message throw and catch events. Shows explicit Kafka topic mappings for message-based collaboration.
<code>invoice_signal_exchange.bpmn</code>	Intermediate signal throw and catch events. Useful for contrasting signal semantics with the more direct message-event mapping.
<code>invoice_review_deadline.bpmn</code>	Interrupting timer boundary on a task. Demonstrates timeout handling and normal-flow suppression after deadline expiry.
<code>invoice_review_reminder_non_interrupting.bpmn</code>	Non-interrupting timer boundary on a task. Demonstrates reminder-style branching without cancelling the main task.
<code>invoice_processing_error.bpmn</code>	Interrupting error boundary on a task, driven by task failure events from the generated runtime.
<code>invoice_review_escalation.bpmn</code>	Interrupting escalation boundary on a task. Shows how escalation differs from failure as an explicit lifecycle outcome.
<code>invoice_call_activity.bpmn</code>	Call-activity request and reply mapping. Demonstrates the generated call and reply topics plus manual completion of the invoked activity.
<code>invoice_review_subprocess.bpmn</code>	Embedded subprocess with explicit entry and completion handlers. This is the basic scope-aware subprocess sample.
<code>invoice_nested_subprocess.bpmn</code>	Nested embedded subprocesses. Useful for understanding recursive scope generation and subprocess-specific input and output channels.

Model	Primary focus
invoice_subprocess_deadline.bpmn	Interrupting timer boundary attached to a subprocess scope. Demonstrates cancellation at the subprocess level rather than at one task.
invoice_subprocess_reminder_non_interrupting.bpmn	Non-interrupting timer boundary attached to a subprocess scope. Shows branch-without-cancel behavior for a whole scope.
invoice_subprocess_error.bpmn	Interrupting error boundary attached to a subprocess scope. Demonstrates subprocess-level failure propagation.
invoice_event_subprocess_message.bpmn	Non-interrupting embedded event subprocess with message start. Shows side-flow activation within an enclosing subprocess scope.
invoice_event_subprocess_interrupting_message.bpmn	Interrupting embedded event subprocess with message start. Demonstrates cancellation of the enclosing scope before the event subprocess branch starts.
invoice_event_subprocess_timer.bpmn	Embedded event subprocess with timer start. Demonstrates scoped delayed activation tied to the enclosing subprocess lifecycle.
invoice_event_subprocess_error.bpmn	Embedded event subprocess with error start. Demonstrates start-on-failure semantics driven by the canonical lifecycle stream.
invoice_event_subprocess_escalation.bpmn	Embedded event subprocess with escalation start. Demonstrates start-on-escalation semantics within a subprocess scope.

Taken together, the sample set functions as a compact executable coverage matrix. When a new BPMN function is added to the generator, adding a dedicated sample model alongside it has become the preferred way to document the feature, test the generator, and provide a runnable teaching example.

22 Example Walkthrough

22.1 Chosen example

The simplest useful walkthrough model in the repository is `src/test/resources/bpmn/invoice_receipt.bpmn`. It contains a start event, service work, a review step, an approval or rejection path, and a final notification step. That makes it a good reference model because it is small, but still representative of the core generator path.

22.2 Generation step

The normal way to generate the project is:

- `mvn-qcleanpackage`
- `java-jartarget/durga-1.0-SNAPSHOT.jar<bpmn-file>`

For this walkthrough, use the repository sample:

```
src/test/resources/bpmn/invoice_receipt.bpmn
```

By default the output is written under `generated/`. The generated project contains Java sources, a Maven build, Kafka topic declarations, helper scripts and a generated README.

22.3 What appears in the generated project

For a model like `invoice_receipt`, the generated output typically includes:

- worker services for automatic task execution,
- waiting-state handlers for any human-completion steps,
- a starter class that puts the first process event onto the start topic,
- a process orchestrator that detects terminal completions,
- helper publishers and watchers for demos and manual testing,
- generated Kafka bindings in `application.yml`.

The resulting project is intentionally explicit. Rather than hiding the process behind one entry point, Durga leaves the execution topics, task handlers and probes visible.

22.4 Runtime shape

For this example, the generated runtime follows a predictable pattern:

1. a starter publishes the initial process event,
2. generated task workers or waiting handlers consume task input topics,
3. each step emits lifecycle events to `process-events`,
4. routing services move the process to the next task or end event,

5. the orchestrator emits explicit process completion.

The same pattern scales outward to the richer BPMN samples in the repository. More advanced models add boundaries, subprocess scopes and event subprocesses, but they still rely on the same basic discipline: generated handlers move the process through Kafka topics while the canonical lifecycle stream captures the truth needed for monitoring.

22.5 How to explore the generated process

The generated helper scripts are the fastest way to understand the runtime:

- `demo-scenario.sh` exercises a whole process path,
- `send-task-input.sh` pushes one specific task input,
- `complete-task.sh`, `fail-task.sh` and `escalate-task.sh` let the user force specific task outcomes,
- `watch-process-events.sh` and `watch-task-output.sh` expose the generated flow as it runs.

This is a central idea in Durga. Generated projects are not only meant to compile; they are meant to be played with and observed.

23 Current Status

23.1 Maturity

Durga is in a strong experimental or prototype state rather than in a finished engine state. The generator is materially capable, the repository now has meaningful regression coverage, and the generated projects are usable for local exploration. At the same time, the project should still be understood as a controlled executable subset rather than a complete BPMN runtime.

23.2 What is solid today

Several parts of the system are now in reasonably good shape:

- the generator has been refactored away from a single large class and a single large template file,
- dry-run and generated-project compile tests cover representative BPMN features,
- the core runtime contracts (`ProcessEvent`, `ProcessState`, `ScopeCancellationRegistry`) are covered by dedicated unit tests,
- the monitoring path is coherent and directly runnable,
- cancellation behavior for interrupting boundaries now affects runtime behavior through a plain in-memory registry rather than an unused CDI annotation,
- demo and probe tooling are generated as part of the process output,
- a GitHub Actions CI pipeline runs the full test suite on every push and pull request,
- BPMN sample fixtures have been moved to `src/test/resources/bpmn/`,
- `SendTask`, `ReceiveTask`, `ScriptTask` and `BusinessRuleTask` now generate dedicated handlers instead of silently falling through to generic auto-completing workers,
- inclusive gateway (OR) split and join are now supported alongside XOR and AND,
- multi-instance loop characteristics are detected and reported in generated summaries.

23.3 Important limits

The present limits are mostly about semantics, not build hygiene:

- support is still a BPMN subset,
- some advanced scope semantics remain approximated rather than fully modeled,
- event subprocess support is deliberately scoped and not yet generalized to all top-level process cases,
- query and monitoring behavior are strong enough for local understanding, but not yet a complete production operations surface.

23.4 Likely next decisions

The next architectural decisions are not about whether the generator works. They are about where the support boundary should stop:

Top-level event subprocess semantics It remains an open design choice whether timer, error and escalation event subprocess starts should be given first-class process-level semantics.

Coverage breadth Inclusive and complex gateways remain unsupported.

Runtime fidelity There is still room to deepen true BPMN scope semantics if the project evolves from exploration tool toward a more complete runtime.

23.5 Practical summary

Durga is already useful as a technical exploration tool for BPMN-to-Kafka mapping. It is able to generate, run, observe and test a substantial set of workflow constructs locally. That is the right frame for the current manual as well: this document should be read as system documentation for an actively evolving prototype with a real executable surface.

24 Glossary

BPMN Business Process Model and Notation, the process-modeling notation used as input to Durga.

Canonical lifecycle stream The shared `process-events` topic that carries normalized process and activity events.

Call activity A BPMN element used to model invocation of another process-like unit, mapped in Durga through explicit call and reply topics.

Event subprocess A subprocess triggered by an event within an enclosing scope instead of by the main sequence flow.

Execution topics The generated Kafka topics that move work between tasks, gateways and handlers.

Generated helper scripts The scaffolded scripts used to run demos, inject inputs, complete tasks, force failures and observe output.

Kafka Streams state store The local materialized store used by the monitoring app to answer low-latency queries.

Process orchestrator The generated runtime class that detects terminal flows and emits explicit completion for the enclosing process.

ProcessStateView The repository's materialized per-instance monitoring projection.

Scope cancellation The generated runtime behavior that suppresses or redirects normal flow after an interrupting boundary or interrupting event subprocess fires.

Scaffolder The BPMN-driven generator entry point, implemented by:

```
org.gautelis.durga.tools.BpmnScaffolder
```

Subprocess scope service A generated class that manages entry, completion and scope-level lifecycle behavior for an embedded subprocess.